CASE STUDY 01 • POSTGRADUATE CYBERSECURITY

OWASP Juice Shop Web Application Security Assessment

A concise, evidence-led adaptation of a 58-page academic security report • Lab work documented in May 2026

EXECUTIVE SUMMARY

Assessed five OWASP Top 10 (2021) risk areas in a deliberately vulnerable local OWASP Juice Shop environment. The work combined practical browser-based tests, HTTP inspection, analysis of likely weaknesses and prevention recommendations. This public summary reports what the original screenshots and written test notes establish—and separates confirmed outcomes from plausible but unverified attack paths.

Assessment context

Target	OWASP Juice Shop (intentionally vulnerable training application)
Environment	Vagrant + VirtualBox virtual machine; local browser access
Techniques	Manual input testing, registration/recovery flows, browser developer tools, HTTP request/response analysis

Results at a glance

Area	Evidence status	What the source supports
SQL injection	**Demonstrated**	Login challenge completion shown in screenshot.
Broken access control	**Not established**	Authenticated order-history API request observed; no cross-user test.
Security misconfiguration	**Observed**	API metadata and verbose Express error response exposed.
Authentication failures	**Demonstrated**	Weak-password account creation and password reset documented.
SSRF	**Not established**	URL submission and 302 response observed; no internal fetch proved.

EVIDENCE RULE

“Demonstrated” means a practical outcome is supported by the original records; “Observed” means a response or behaviour was captured, but not necessarily an exploitable flaw; “Not established” means the experiment does not prove the specific vulnerability. These statuses describe evidence strength, not CVSS ratings.

My practical contribution

I set up the Vagrant / VirtualBox lab, exercised login and account-recovery workflows, inspected application traffic and response data in browser developer tools, and documented what each observation did—and did not—prove. The lab work and screenshots originate in my May 2026 coursework.

TESTS 01–02 • VERIFIED LAB OUTCOMES

Injection & Authentication

Selected screenshots reproduced from the original report; this portfolio version contains cropped or anonymised copies.

01 / SQL injection — authentication-bypass challenge

I entered an SQL-style payload into the Juice Shop login form and submitted the attempt. The subsequent screenshot shows its green “Login Admin” challenge-completion banner. The original report documents the interaction in section 2.3; the result belongs to an intentionally vulnerable training target, not a production system.

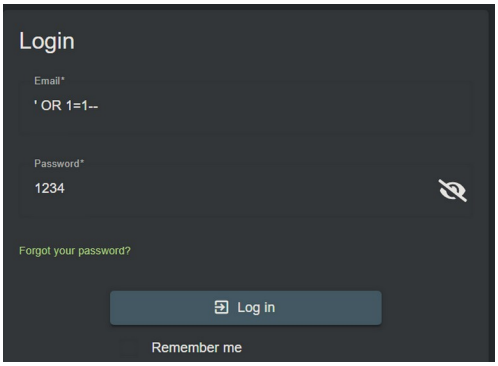


Fig. 1 — Login-form input (original report, p. 9).

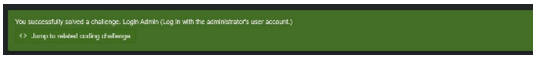


Fig. 2 — “Login Admin” challenge-solved banner (p. 10).

Why it matters: Unsafe query construction can alter authentication logic. Parameterised queries are the primary recommended control; input validation and least-privileged database access add defence in depth.

Limit: UNION-based extraction is discussed in the source report as an alternative scenario, but no extraction outcome is documented.

02 / Authentication — weak passwords and account recovery

I created a lab account using “12345” (a weak five-character password) and then reset the account password to “123456” using its security question. The report records both completed workflows; the cropped registration and reset screens support the surrounding evidence. This is not a demonstration of taking over another person’s account.

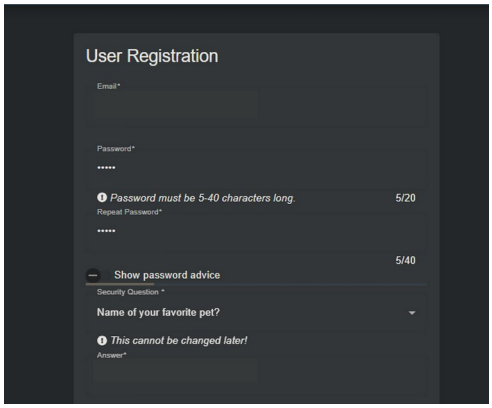


Fig. 3 — Registration input (p. 36; test details described in text).

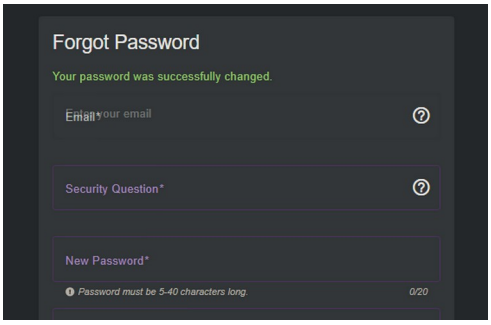


Fig. 4 — Successful password-reset message (p. 39).

Recommended controls: Block commonly used passwords, support long passphrases, protect recovery with time-limited tokens and appropriate verification, and apply rate limiting / MFA where relevant.

TESTS 03–04 • HTTP INSPECTION

Access Control & Information Exposure

The key analytical distinction: observing an endpoint is not the same as bypassing authorisation.

03 / Broken access control — authenticated request inspection

I opened the authenticated order-history page and captured its /rest/order-history traffic using the Network tab. This confirms client–server communication with an authenticated request, but the original report does not include a second-user or cross-account test. An authorisation flaw therefore remains unverified.

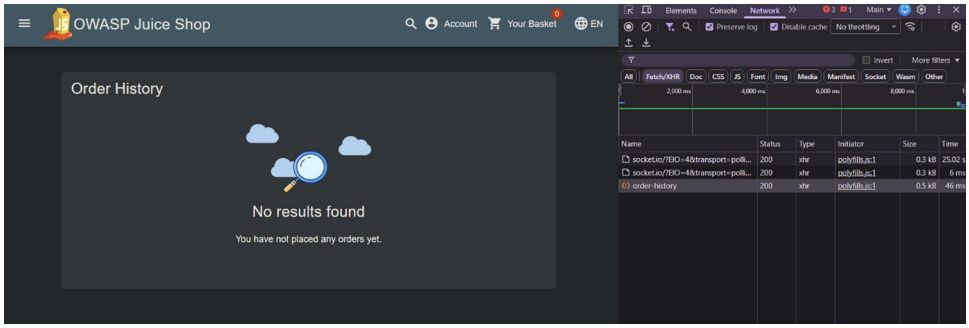


Fig. 5 — Order-history request capture (pp. 17–18).

Next test—not performed: Compare responses for two authorised lab identities and vary resource ownership to determine whether the server independently enforces object-level access.

04 / Security misconfiguration — information disclosure

I inspected a product API JSON response containing createdAt, updatedAt and deletedAt fields and observed a separate HTTP 500 error disclosing Express 4.22.1, an internal route and stack-trace information. These are directly visible in the source screenshots; the material does not prove access to confidential records or code execution.

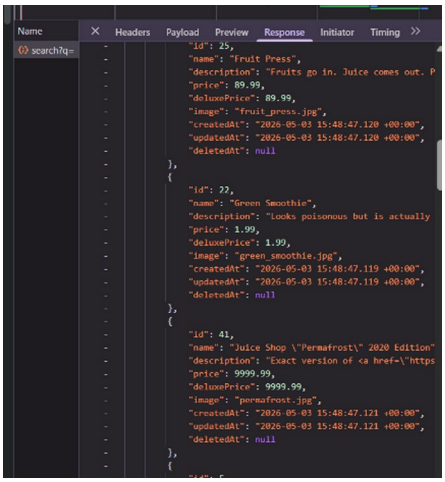


Fig. 6 — JSON metadata: createdAt / updatedAt / deletedAt (p. 27).

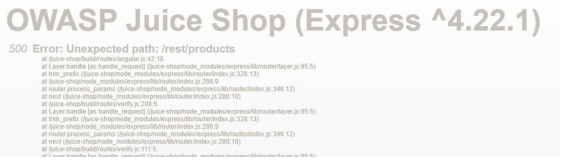


Fig. 7 — HTTP 500, Express 4.22.1 and stack trace (p. 27).

Evidence readout: Product JSON includes createdAt, updatedAt, deletedAt. The error page shows “500 Error: Unexpected path: /rest/products” and “Express ^4.22.1”. Recommended controls: return only necessary JSON fields and replace detailed production errors with generic responses.

TEST 05 • LIMITED OBSERVATION

Server-Side Request Forgery (SSRF)

A bounded test of URL-handling functionality, not a confirmed SSRF exploit.

What was submitted

I entered a lab URL in the profile image field and inspected the Network panel after submission. The browser sent POST /profile/image/url and received HTTP 302 Found. This proves endpoint submission and redirection only; no outbound request from the server, internal-service connection or data retrieval was shown.

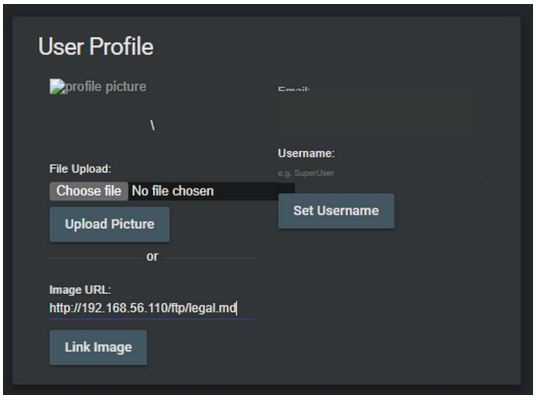


Fig. 8 — Profile image URL submission (p. 48).

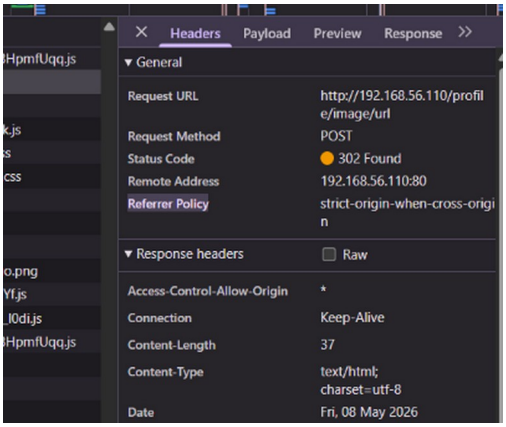


Fig. 9 — POST /profile/image/url; HTTP 302 Found (p. 49).

OBSERVED OUTCOME

The browser submitted a URL and received a response from the application endpoint. This is direct evidence of application URL-submission handling and redirect behaviour.

NOT DEMONSTRATED

The source does not establish that the server fetched the submitted target, reached an internal service, accessed protected data or returned internal content. A browser-to-application POST is not, by itself, proof of server-side request forgery. The HTTP 302 response does not resolve that missing evidence.

Security analysis and possible next validation

Features that fetch URLs on a user's behalf can create SSRF risk if destinations are not restricted. To verify SSRF, a separately authorised lab test would require an observable destination (for example, a controlled callback service or server-side request logs), confirmation of an outbound request initiated by the application, and evidence that safeguards can be bypassed. None of those follow-up tests are claimed here.

Recommended controls: Allow-list approved URL destinations and protocols, reject loopback / private / link-local targets, revalidate after redirects, and constrain outbound network access.

SYNTHESIS • TRANSFERABLE SKILLS

Interpretation, Recommendations & Evidence

The purpose of this case study is to make the testing process inspectable and the conclusions defensible.

Prioritised remediation recommendations

#	Control	Recommended action
01	**Prevent SQL injection**	Use parameterised queries; test authentication code paths and use minimally privileged database identities.
02	**Harden authentication**	Reject common passwords; secure account recovery, support MFA and apply rate limits.
03	**Validate authorisation**	Check per-user / per-object permissions at the API layer; test with separate lab identities.
04	**Reduce information leakage**	Return only needed JSON properties; suppress detailed production error traces and log them privately.
05	**Constrain outbound URL handling**	Allow-list destinations; block internal addresses and enforce egress controls. Further testing needed to establish SSRF.

What the work demonstrates

Virtualised lab setup: I configured a Vagrant / VirtualBox environment for the OWASP Juice Shop training application.

Practical test execution: I entered lab test inputs, completed account registration and recovery exercises, and monitored browser requests.

Evidence analysis: I interpreted a challenge-completion banner, API responses, status codes and error traces, while flagging unverified access-control and SSRF hypotheses.

Technical reporting: I documented steps, observed outcomes, testing limitations and suggested controls across five risk areas.

Boundaries and transparency

This was academic lab work, not a professional penetration test. The original 58-page report is a private source document rather than a public attachment. The adaptation uses selected original screenshots, conceals visible lab account identifiers or answers, and omits student details. Findings describe a deliberately vulnerable training application; results are not generalised to a production target. No additional hands-on testing or remediation verification was performed while creating this portfolio summary.

Evidence map & source provenance

Source: “OWASP Report” (May 2026), Web Application Security and Software Standards module. All screenshots derive from this 58-page academic assignment. The table below maps the portfolio figures to the original numbered pages; exact file names and boundaries are recorded in EVIDENCE_REVIEW.md.

Risk area	Figures	Source report
SQL injection	1–2	pp. 9–10, §2.3
Authentication	3–4	pp. 36–39, §5.3
Access control	5	pp. 17–18, §3.3
Misconfiguration	6–7	p. 27, §4.3
SSRF	8–9	pp. 48–49, §6.3

PUBLISH WITH CONTEXT

Suggested repository title: owasp-juice-shop-security-assessment. Publish the case study PDF, README and sanitised evidence files. Keep the full original academic document private until you have independently reviewed it for personal information and any academic reuse restrictions.